

retouch-gem — Product Requirements Document

Version: 1.0 **Date:** 2026-02-23 **Author:** PRD Generator **Status:** Draft

Executive Summary

retouch-gem is a jewelry virtual try-on web application that lets customers of e-commerce jewelry brands visualize how a gem or jewelry piece looks realistically composited onto a hand model. Businesses share a branded link; customers browse the product catalog or upload their own jewelry photo, select from pre-set hand models, and Google Gemini AI generates a photorealistic try-on result in 5–15 seconds. The product targets online-first jewelry brands who want to reduce purchase hesitation, increase conversion rates, and eliminate the need for per-combination product photography. Success is measured through customer engagement metrics — try-on session rate, result downloads, and completed try-on flows.

Problem Statement

Current state: E-commerce jewelry brands rely on static product photography on white backgrounds, giving customers no way to visualize scale, fit, or how a piece looks when actually worn. Physical try-on is impossible online, and custom hand-model photography for every product combination is expensive and slow.

Pain points:

- 1. Customers cannot visualize how a ring or bracelet looks at scale on a real hand
- 2. High return rates driven by "didn't look as expected" post-purchase disappointment
- 3. Per-combination hand-model photography costs thousands of dollars and weeks of production
- 4. No interactive way to compare jewelry pieces across consistent hand models
- 5. Competitors offering AR or AI try-on have a meaningful engagement advantage

Impact: Higher cart abandonment, elevated return rates, and missed conversion opportunities for brands lacking immersive product experiences. Estimates suggest 30–40% of jewelry returns are fit/aesthetic-related.

Goals & Success Metrics

Goal	Metric	Target	Measurement Method
Drive try-on engagement	Try-on sessions / unique visitors	> 40%	Frontend analytics
Increase result downloads	Downloads per completed try-on	> 25%	Backend download tracking
Minimize AI failures	Gemini error rate	< 5%	Backend error logging
Fast turnaround	Gemini processing time (P95)	< 15 seconds	Backend latency metrics
Easy admin onboarding	Time to upload first product + model	< 5 minutes	UX target

Maintain quality	Try-on completion rate (no abandonment mid-flow)	> 70%	Frontend funnel analytics
------------------	--	-------	---------------------------

User Personas

Persona 1: Online Jewelry Shopper (Mia)

- **Role:** Customer browsing a jewelry brand's try-on link
- **Goals:** Visualize jewelry before purchasing; find a piece that complements her hand
- **Pain points:** Flat product photos don't convey scale or real-world aesthetics; worried about expensive returns
- **Technical proficiency:** Medium — comfortable with web apps and basic file uploads
- **Usage context:** Mobile or desktop, during a shopping session; low patience for slow or confusing UI

Persona 2: E-Commerce Brand Manager (James)

- **Role:** Admin managing the jewelry catalog, hand models, and Gemini prompt configuration
- **Goals:** Quickly add new products and models; tune prompts to improve result quality
- **Pain points:** Traditional product photography is slow and expensive; wants a self-serve tool
- **Technical proficiency:** Medium — comfortable with web dashboards, not a developer
- **Usage context:** Desktop, periodic product updates, prompt experimentation

Persona 3: Developer / Technical Owner (Dev)

- **Role:** Deploying and maintaining retouch-gem for a brand
- **Goals:** Clean API contracts, reliable Gemini integration, fast local setup
- **Pain points:** Unclear integration points, opaque error messages, complex env configuration
- **Technical proficiency:** High — full-stack developer
- **Usage context:** Initial setup, environment configuration, debugging, future migrations

Functional Requirements

Customer-Facing Features

FR-001: Product Catalog Browse

Description: Customers can view all jewelry products uploaded by the admin in a responsive gallery.

User story: As a customer, I want to browse available jewelry products so that I can select one to try on.

Acceptance criteria:

- ☐ Products display as a responsive grid with product image and name
- ☐ Grid loads in < 1.5 seconds for up to 50 products
- ☐ Clicking a product card selects it and shows a visual highlight
- ☐ Empty state shows a message when no products are uploaded
- ☐ Grid is paginated or scrollable without performance degradation up to 200 products

Priority: P0

FR-002: Customer Own Jewelry Upload

Description: Customers can upload their own jewelry photo to use in the try-on instead of a catalog product.

User story: As a customer, I want to upload my own jewelry image so that I can try on pieces I already own or found elsewhere.

Acceptance criteria:

- ☐ Upload accepts JPEG, PNG, WEBP formats up to 10 MB
- ☐ Client-side image preview shown immediately after file selection
- ☐ Custom upload replaces any previously selected catalog product
- ☐ Invalid file type returns a clear, user-friendly error message
- ☐ File size exceeded returns a clear error message before upload attempt

Priority: P0

FR-003: Hand Model Gallery Selection

Description: Customers can browse and select from pre-set hand model photos uploaded by the admin.

User story: As a customer, I want to choose a hand model so that I can see the jewelry on a hand that best represents me.

Acceptance criteria:

- ☐ Hand models display in a gallery grid with model name revealed on hover
- ☐ Selected model is highlighted with a border or checkmark indicator
- ☐ At least 1 hand model must exist before the try-on flow allows submission
- ☐ Gallery supports up to 20 hand models without layout or performance degradation

Priority: P0

FR-004: Try-On Execution

Description: Customers trigger the Gemini AI service to composite their selected jewelry onto the chosen hand model.

User story: As a customer, I want to click "Try On" and receive a realistic result image so that I can evaluate how the jewelry looks.

Acceptance criteria:

- ☐ "Try On" button is disabled until both a jewelry item and a hand model are selected
- ☐ Loading state displays an elegant shimmer/pulse animation during processing
- ☐ Result is displayed within 15 seconds in $\geq 95\%$ of cases
- ☐ Processing timeout at 30 seconds returns a user-friendly "please try again" message
- ☐ Cached results (same catalog product + hand model) load in < 500 ms
- ☐ Error states show a non-technical message with a retry action

Priority: P0

FR-005: Result Viewer and Download

Description: Customers can view the generated try-on result image and download it.

User story: As a customer, I want to view and download the try-on result so that I can save or share it.

Acceptance criteria:

- ☐ Result image appears with a smooth fade-in reveal animation
- ☐ "Download" button saves the image as JPEG or PNG to the user's device
- ☐ "Try Another" button resets to hand model selection without refreshing the page
- ☐ Result image is minimum 1024 px on the short side
- ☐ Result URL is stable and shareable

Priority: P0

Admin Features

FR-006: Admin Authentication

Description: The `/admin` route is protected by a hardcoded API key stored in an environment variable.

User story: As an admin, I want a protected dashboard so that only I can manage content.

Acceptance criteria:

- ☐ `/admin` and all sub-routes redirect to `/admin/login` if unauthenticated
- ☐ Admin key configured via `ADMIN_PASSWORD` environment variable
- ☐ Failed login shows an error message; no route access granted
- ☐ Successful login stores a session token in `localStorage` valid for 24 hours
- ☐ Logout clears the session token and redirects to `/admin/login`

Priority: P0

FR-007: Product Management

Description: Admin can upload, view, and delete jewelry products.

User story: As an admin, I want to manage the product catalog so that customers see the latest jewelry collection.

Acceptance criteria:

- ☐ Upload form accepts product name (required, max 100 chars) + image file
- ☐ Accepted image formats: JPEG, PNG, WEBP; max file size 10 MB
- ☐ Uploaded products appear in the admin list and customer catalog immediately
- ☐ Delete action requires confirmation dialog before removal
- ☐ Deleted product is removed from catalog; existing cached results using it are preserved
- ☐ Up to 200 products supported without list performance degradation

Priority: P0

FR-008: Hand Model Management

Description: Admin can upload, view, and delete hand model photos.

User story: As an admin, I want to manage hand models so that customers have diverse options for visualizing jewelry.

Acceptance criteria:

- ☐ Upload form accepts model name (required, max 100 chars) + image file
- ☐ Accepted formats: JPEG, PNG, WEBP; max file size 15 MB
- ☐ Uploaded models appear in admin list and customer gallery immediately
- ☐ Delete action requires confirmation dialog
- ☐ Up to 20 hand models supported

Priority: P0

FR-009: Prompt Template Management

Description: Admin can create, edit, and set a default Gemini prompt template used for try-on generation.

User story: As an admin, I want to tune the Gemini prompt so that I can improve try-on result quality over time.

Acceptance criteria:

- ☐ Admin can view all prompt templates in a list with name and preview
- ☐ Create new prompt with name and body textarea (no character limit)
- ☐ Edit existing prompt's name and body
- ☐ Set any prompt as default; only one prompt can be default at a time
- ☐ Default prompt is used for all try-on generations automatically
- ☐ System ships with a pre-filled default prompt on first launch
- ☐ Deleting the default prompt is blocked unless another default is set first

Priority: P1

FR-010: Result Cache Invalidation

Description: Admin can force regeneration of cached try-on results after prompt updates.

User story: As an admin, I want to clear cached results so that customers see fresh results after I update a prompt.

Acceptance criteria:

- ☐ Admin can trigger "Clear All Cache" action from the dashboard
- ☐ Individual cache entries are keyed by `product_id + hand_model_id`
- ☐ Forced regeneration deletes stored result files and recalculates on next customer request

Priority: P2

Non-Functional Requirements

Performance

- Customer page initial load: < 2 seconds on a 4G connection (Vercel CDN-served)
- Gemini try-on processing: < 15 seconds at P95; hard timeout at 30 seconds
- Cached try-on response: < 500 ms
- All non-Gemini API endpoints: < 200 ms response time

- File upload endpoints: handle up to 15 MB multipart with streaming (no buffering full body in memory)

Security

- Admin key stored exclusively in `.env` ; never bundled in client-side code
- File uploads validated for MIME type server-side before storage
- Uploaded filenames replaced with UUID-based names to prevent path traversal
- CORS configured to allow only the frontend origin and `localhost:3000`
- No customer PII collected or stored in MVP

Scalability (MVP)

- Target: single business, 10–100 concurrent customers
- File storage designed for local-to-S3 migration (paths abstracted)
- Gemini API: single API key; no rate-limit handling for MVP

Availability

- Uptime target: 99% best-effort for MVP (Railway/Render standard SLA)
- No complex failover or disaster recovery required for MVP

Technical Architecture

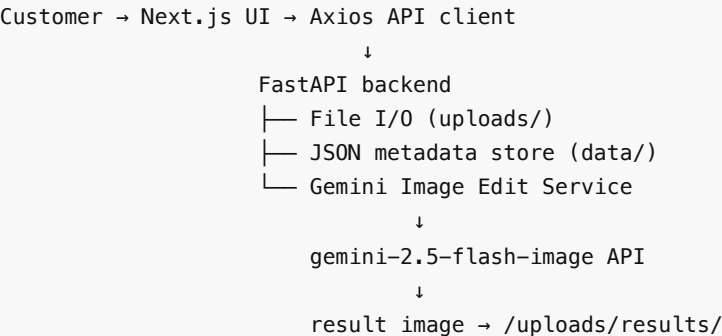
System Overview

Next.js frontend (served from Vercel) communicates with a FastAPI backend (hosted on Railway/Render) via REST API. The backend handles file uploads, stores metadata as JSON files, calls the Gemini AI service for image compositing, caches results by product+model combination, and serves static files from a local `/uploads/` directory.

Technology Stack

- **Frontend:** Next.js 14+ (App Router), React 18, TypeScript, Tailwind CSS
- **Backend:** FastAPI, Python 3.11+, Uvicorn
- **AI:** Google Gen AI SDK (`google-generativeai`), model `gemini-2.5-flash-image`
- **Image processing:** Pillow (PIL)
- **Metadata store:** JSON files (`data/*.json`) for MVP
- **File storage:** Local filesystem (`/uploads/`) → Cloudflare R2/S3 (post-MVP)
- **Hosting:** Vercel (frontend) + Railway or Render (backend)

Architecture Data Flow



↓
result_url → frontend display

API Specifications

GET /api/products

Purpose: List all products in the catalog. **Authentication:** None

Response (200):

```
[
  {
    "id": "uuid",
    "name": "string",
    "image_url": "/uploads/products/uuid.jpg",
    "created_at": "2026-02-23T10:00:00Z"
  }
]
```

POST /api/products

Purpose: Upload a new jewelry product. **Authentication:** Required — X-Admin-Key header

Request: multipart/form-data

```
name: string (required)
image: file (JPEG/PNG/WEBP, max 10 MB)
```

Response (201):

```
{
  "id": "uuid",
  "name": "string",
  "image_url": "/uploads/products/uuid.jpg",
  "created_at": "2026-02-23T10:00:00Z"
}
```

Error responses:

- 400: Missing name , invalid file type, or file exceeds size limit
 - 401: Invalid or missing admin key
-

DELETE /api/products/{id}

Purpose: Delete a product by ID. **Authentication:** Required — X-Admin-Key header

Response (204): No content

Error responses:

- 401: Invalid admin key

- 404: Product not found

GET /api/hand-models

Purpose: List all active hand models. **Authentication:** None

Response (200):

```
[
  {
    "id": "uuid",
    "name": "string",
    "image_url": "/uploads/hand-models/uuid.jpg"
  }
]
```

POST /api/hand-models

Purpose: Upload a new hand model. **Authentication:** Required — X-Admin-Key header

Request: multipart/form-data

```
name: string (required)
image: file (JPEG/PNG/WEBP, max 15 MB)
```

Response (201): Hand model object (same shape as list item)

Error responses:

- 400: Missing fields, invalid file type, or file exceeds size
- 401: Invalid admin key

DELETE /api/hand-models/{id}

Authentication: Required — X-Admin-Key header **Response (204):** No content **Error responses:** 401, 404

GET /api/prompts

Purpose: List all prompt templates. **Authentication:** Required — X-Admin-Key header

Response (200):

```
[
  {
    "id": "uuid",
    "name": "string",
    "body": "string",
    "is_default": true,
    "created_at": "2026-02-23T10:00:00Z"
  }
]
```

```
}  
]
```

POST /api/prompts

Authentication: Required — X-Admin-Key header

Request:

```
{  
  "name": "string",  
  "body": "string"  
}
```

Response (201): Prompt object

PUT /api/prompts/{id}

Authentication: Required — X-Admin-Key header

Request:

```
{  
  "name": "string",  
  "body": "string"  
}
```

Response (200): Updated prompt object

PUT /api/prompts/{id}/default

Purpose: Set this prompt as the default for all try-ons. **Authentication:** Required — X-Admin-Key header

Response (200):

```
{  
  "id": "uuid",  
  "name": "string",  
  "body": "string",  
  "is_default": true  
}
```

Error responses:

- 404: Prompt not found
-

POST /api/try-on

Purpose: Generate a try-on result via Gemini AI. **Authentication:** None (customer-facing)

Request (catalog product — JSON):

```
{
  "product_id": "uuid",
  "hand_model_id": "uuid"
}
```

Request (custom upload — multipart/form-data):

```
jewellery_image: file (JPEG/PNG/WEBP, max 10 MB)
hand_model_id: string (UUID)
```

Response (200):

```
{
  "result_url": "/uploads/results/uuid.jpg",
  "cached": false,
  "processing_time_ms": 8500
}
```

Error responses:

- 400: Invalid product/hand model ID, missing fields, or invalid file
- 404: Product or hand model not found
- 408: Gemini processing timeout (>30s)
- 500: Gemini API error or returned no image

GET /uploads/{path}

Purpose: Serve uploaded images (products, hand models, results) as static files. **Authentication:** None

Response: Image binary (JPEG/PNG/WEBP)

UI/UX Requirements

Design Language: Luxury editorial — deep charcoal background (#0d0d0d), warm gold accents (#c9a84c), off-white text (#f5f0e8), Cormorant Garamond display font, DM Sans body font. Motion: staggered fade-in reveals, smooth image transitions.

Customer Landing Page

Purpose: First impression, brand showcase, and entry point to the try-on experience.

Key elements:

- Full-bleed hero with animated jewelry imagery and headline copy
- "Try On Now" CTA scrolls to product catalog
- Minimal navigation (brand logo only)

User flow:

1. Customer lands via shared link
2. Sees hero animation and CTA
3. Scrolls to product gallery

States:

- Loading: skeleton cards for product grid
 - Empty: "No products available" message
-

Try-On Experience Page (/try-on)

Purpose: Core interaction — select jewelry, select hand model, generate and view result.

Key elements:

- **Step 1 — Jewelry:** Catalog grid with "Upload your own" zone below
- **Step 2 — Hand Model:** Gallery with hover-reveal names
- **Step 3 — Generate:** "Try On" CTA (disabled until both selections are made)
- **Loading state:** Shimmer/pulse animation covering result area
- **Result panel:** Fade-in result image + Download button + "Try Another" link

User flow:

1. Customer selects jewelry from catalog or uploads image
2. Customer selects hand model from gallery
3. "Try On" button activates; customer clicks it
4. Loading animation plays (~5–15s)
5. Result fades in; customer downloads or retries

States:

- Default: Step 1 highlighted; steps 2–3 visually muted
 - Step 1 complete: Step 2 becomes active
 - Both complete: "Try On" button activates
 - Processing: shimmer animation, button disabled
 - Result: result image + action buttons
 - Error: inline error message with retry button
-

Admin Login (/admin/login)

Key elements: Centered card with password input and "Login" button.

States:

- Default: empty form
 - Error: "Invalid admin key" inline message
-

Admin Dashboard (/admin)

Purpose: Content management hub.

Key elements:

- Sidebar navigation: Products | Hand Models | Prompts
 - Quick stats: product count, hand model count, active prompt name
 - Recent activity log (optional, P2)
-

Admin Sub-Pages (Products / Hand Models / Prompts)

Pattern repeated for each:

- Upload form at top (drag-and-drop or file picker)
- Item list below with thumbnail, name, and delete button
- Default prompt indicator (star icon) for Prompts page

Data Models

Product

Field	Type	Required	Description
id	UUID	Yes	Primary key
name	string (max 100)	Yes	Product display name
image_path	string	Yes	Relative path from /uploads/products/
image_url	string	Yes	Public URL for frontend display
created_at	timestamp (UTC)	Yes	Creation timestamp

HandModel

Field	Type	Required	Description
id	UUID	Yes	Primary key
name	string (max 100)	Yes	Model display name
image_path	string	Yes	Relative path from /uploads/hand-models/
image_url	string	Yes	Public URL for frontend display

Prompt

Field	Type	Required	Description
id	UUID	Yes	Primary key
name	string (max 100)	Yes	Template name
body	string	Yes	Full Gemini prompt text
is_default	boolean	Yes	Whether this is the active prompt (only one true)
created_at	timestamp (UTC)	Yes	Creation timestamp

TryOnResult (Cache)

Field	Type	Required	Description
id	UUID	Yes	Primary key

product_id	UUID	No	Null for custom uploads
hand_model_id	UUID	Yes	Reference to HandModel
result_path	string	Yes	Local path to result image
result_url	string	Yes	Public URL for frontend display
created_at	timestamp (UTC)	Yes	UTC creation timestamp

Indexes:

- `(product_id, hand_model_id)` — composite key for cache lookup

Integration Points

Google Gemini AI

Purpose: Core AI image compositing — places jewelry from one image onto a hand model image.

Integration type: Python SDK (`google-generativeai`)

Model: `gemini-2.5-flash-image`

Data exchanged:

- Inbound to Gemini: prompt text + jewelry image (PIL Image) + hand model image (PIL Image)
- Outbound from Gemini: composited result image bytes via `part.inline_data.data`

Authentication: `G00GLE_API_KEY` environment variable, accessed via `generativeai.Client()`

Rate limits: Subject to Google AI API quotas; no custom rate-limiting in MVP

Fallback behavior: If `inline_data` is None (Gemini returns text instead of image), raise `ValueError` and return HTTP 500 with user-friendly message. No partial results stored.

Edge Cases & Error Handling

Scenario	Expected Behavior
Gemini returns no image (text response)	Return 500: "Try-on generation failed — please retry"
Gemini timeout (> 30s)	Return 408: "Processing timed out — please try again"
Upload exceeds size limit	Client validates first; backend returns 400 as fallback
Invalid file type uploaded	Server-side MIME check returns 400 with format list
Same catalog product + hand model requested twice	Return cached result instantly (< 500 ms)
Custom upload (no product_id) requested again	No cache — always calls Gemini

Hand model deleted while customer is mid-flow	Frontend shows 404 on hand model fetch; prompt re-selection
No default prompt set	Use hardcoded fallback prompt string from config
Concurrent try-on requests for same combo	Return first cached result; second call waits or reuses cache
Admin deletes default prompt with no replacement	Block deletion with inline error message

Error Handling Strategy

- **User-facing errors:** Clear, non-technical messages with actionable retry suggestions
 - **System errors:** Full stack traces logged server-side; generic message returned to client
 - **Retry logic:** No automatic retries — customer manually retries after error
 - **Graceful degradation:** If prompt API fails, use hardcoded default prompt and log warning
-

Testing Requirements

Unit Tests

- `gemini_service.py` — mock SDK client, verify correct image bytes and prompt are passed
- File upload validation — test MIME type rejection and size limit enforcement
- Cache key logic — verify `product_id:hand_model_id` composite key generation
- Admin auth dependency — verify 401 returned when key is missing or wrong

Integration Tests

- `POST /api/try-on` with catalog product → 200 with valid `result_url`
- `POST /api/try-on` with custom upload → 200 with valid `result_url`
- `POST /api/products` without admin key → 401
- Full product lifecycle: upload → list → delete → confirm removal
- Full hand model lifecycle: upload → list → delete → confirm removal
- Prompt default switch: set prompt A as default, then set B — verify only B is default

E2E Tests

- Customer complete flow: browse catalog → select hand model → try on → verify result displayed
- Customer upload flow: upload custom image → select hand model → try on → download result
- Admin flow: login → upload product → upload hand model → trigger try-on via customer flow

Performance Tests

- Try-on endpoint: P95 processing time < 15 seconds under single concurrent user
 - Product listing: response < 200 ms for 50 products
 - Cached result: response < 500 ms on second identical request
-

Implementation Notes for AI

Build Order

1. `backend/app/models/schemas.py` — Pydantic models for all entities
2. `backend/app/services/gemini_service.py` — Gemini integration with PIL image handling

3. backend/app/api/products.py — product CRUD with file upload
4. backend/app/api/hand_models.py — hand model CRUD with file upload
5. backend/app/api/prompts.py — prompt CRUD with default management
6. backend/app/api/try_on.py — try-on endpoint with caching logic
7. backend/app/main.py — router registration, CORS, static file mount
8. frontend/src/lib/api.ts — fully typed Axios API client
9. frontend/src/app/page.tsx — customer landing page with product catalog
10. frontend/src/app/try-on/page.tsx — try-on experience page
11. frontend/src/app/admin/page.tsx — admin dashboard
12. frontend/src/app/admin/products/page.tsx — product management
13. frontend/src/app/admin/hand-models/page.tsx — hand model management
14. frontend/src/app/admin/prompts/page.tsx — prompt management

File Structure

```
retouch-gem/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── page.tsx
│   │   │   ├── try-on/page.tsx
│   │   │   ├── admin/
│   │   │   │   ├── login/page.tsx
│   │   │   │   ├── page.tsx
│   │   │   │   ├── products/page.tsx
│   │   │   │   ├── hand-models/page.tsx
│   │   │   │   └── prompts/page.tsx
│   │   │   └── layout.tsx
│   │   ├── components/
│   │   │   ├── ProductCard.tsx
│   │   │   ├── HandModelGallery.tsx
│   │   │   ├── TryOnPanel.tsx
│   │   │   └── ResultViewer.tsx
│   │   └── ui/
│   ├── lib/
│   │   └── api.ts
│   ├── package.json
│   └── tailwind.config.ts
└── backend/
    ├── app/
    │   ├── main.py
    │   ├── config.py
    │   ├── api/
    │   │   ├── products.py
    │   │   ├── hand_models.py
    │   │   ├── prompts.py
    │   │   └── try_on.py
    │   ├── services/
    │   │   └── gemini_service.py
    │   └── models/
```

```

├── schemas.py
├── data/
│   ├── products.json
│   ├── hand_models.json
│   ├── prompts.json
│   └── results_cache.json
├── uploads/
│   ├── products/
│   ├── hand-models/
│   └── results/
└── pyproject.toml

```

Critical Implementation Details

- All uploaded files renamed to UUID-based filenames on save (never use original filename — path traversal risk)
- Gemini results stored as JPEG for smaller file size
- In-memory cache dict for MVP: `cache: dict[str, str]` keyed by `"{product_id}:{hand_model_id}"`, value is `result_url`
- JSON data files loaded at startup and mutated in memory; written to disk on every write operation
- Admin key validated via a FastAPI Depends dependency: `async def verify_admin(x_admin_key: str = Header(...))`
- CORS must be registered before any middleware reading the request body; allow origins from `.env`
- FastAPI `StaticFiles` mounted at `/uploads` pointing to `backend/uploads/` directory
- Set `multipart` max size explicitly to avoid body buffering timeouts on large uploads

Libraries to Use

- `aiofiles` for async file read/write
- `python-multipart` for FastAPI file upload handling
- `Pillow` for image type validation and optional JPEG conversion
- `axios` in Next.js with a typed API client (centralize base URL from env)
- `tailwindcss` for all styling — no component library needed for MVP
- `uuid` (Python stdlib) for file/entity ID generation

Libraries to Avoid

- SQLAlchemy for MVP — JSON file store is sufficient and faster to set up
- Next.js API routes — all API traffic goes to FastAPI backend
- `react-query` or `SWR` for MVP — direct axios calls are simpler at this scale

Common Pitfalls

- Gemini `response.parts` may have no item with `inline_data` — always iterate and check before accessing `.data`
- FastAPI CORS middleware must be added **before** routing middleware; wrong order breaks preflight
- Large image uploads time out if FastAPI `request` body is read naively — use streaming or `UploadFile.read()` directly
- Next.js `'use client'` directive is required on any component using `useState`, `useEffect`, or file inputs
- JSON data store is not thread-safe under concurrent writes — acceptable for MVP single-instance deployment

Code Style Preferences

- Python: type hints on all functions; `async def` for all I/O-bound handlers
- TypeScript: strict mode enabled; use `interface` for object shapes
- No `any` in TypeScript — use `unknown` and narrow types
- Backend filename convention: `snake_case.py` ; frontend components: `PascalCase.tsx`
- Environment variables accessed only via a `config.py` singleton (backend) and `lib/config.ts` (frontend)

Assumed Defaults

- No customer authentication in MVP
- No try-on history per customer
- No S3/R2 migration in MVP (local filesystem only)
- No social sharing features
- Single business deployment per instance (not multi-tenant)